Volume 01 | Startup, FastAPI Registration, and Page Routes

Trace the app import, ASGI server, route decorators, and HTML endpoints.

1. Runtime entry point

`Procfile` starts `uvicorn run:app --host 0.0.0.0 --port $PORT`. Uvicorn imports module `run` and reads its `app` object. `run.py` imports `app` from package `app`, imports Uvicorn, and when run directly selects `PORT` (default 5000) and optional `UVICORN_RELOAD` before calling `uvicorn.run('app:app', ...)`. The production Procfile uses Render's environment port; the local fallback is not a fixed service port.

2. Application construction and circular-looking import

`app/__init__.py` calculates `BASE_DIR`, constructs `FastAPI(title, description, version, docs_url='/api/docs')`, and mounts `/static`. It then imports `app.routes`. `routes.py` imports the already-created `app` object and registers handlers using `@app.get`, `@app.post`, and `@app.api_route`. This order works because the object exists before routes import. It is import-time registration, not a request-time route build.

Static directory must exist for `StaticFiles` mount. `routes.py` then reads station CSV, builds `ALL_STATIONS`, and constructs `travel_graph` at module import. `forecasting.py` and `scheduling_optimizer.py` also load model and operational files when imported. Thus, missing/corrupt artifacts can prevent the ASGI app from finishing import and becoming ready.

3. Route map

- `GET /`: landing page, date in Asia/Kolkata, Rajiv Chowk preview profile and model summary.
- `GET /dashboard`: initial dashboard template, station metadata, current date defaults, and default Rajiv Chowk.
- `POST /get_footfall`: accepts HTML form data; awaits `request.form()` then offloads synchronous result rendering to Starlette's threadpool.
- `GET/POST /api/predict`: JSON endpoint; GET reads query values; POST parses JSON when content type includes application/json, otherwise query params.
- `GET /api/stations`: serializes current station metadata.
- `GET /api/model_metrics`: returns model metrics with synthetic-only provenance.
- `/api/docs`: OpenAPI UI generated by FastAPI.

4. Page rendering

The routes use `Jinja2Templates` rooted in `app/templates`. `TemplateResponse(request=request, name=..., context=...)` returns HTML. `landing` computes preview data for a specific station/current date; dashboard sends all station metadata as context. Static JavaScript is not inserted in route code; templates load scripts and the browser handles interactions.

5. Sync and async choices

The endpoint functions that call templates/CSV iteration can be `def`; FastAPI runs synchronous endpoints in its worker pool. `/api/predict` is `async` because it awaits JSON parsing and then explicitly delegates `_predict_data` with `run_in_threadpool`. This is important because Pandas and sklearn prediction code is synchronous. Making a function `async` alone would not yield during a blocking `.predict()` call.

6. Error boundaries and improvements

The app creates JSON 400 responses for malformed/non-object JSON and invalid date/time strings. It does not use a Pydantic input schema for `api_predict`; station validation is incomplete. A clean contract should validate station against canonical metadata, constrain date/time format, specify accepted line values, and return consistent 400/404/422 responses. Add lifespan/startup validation to report which artifact failed, rather than a raw import traceback.

Walkthrough exercise

Explain the sequence of `uvicorn run:app`: module `run` imports package app; FastAPI object constructed; routes imported and register; modules load data/build graph; Uvicorn begins serving. Identify at least four assets that can fail startup and one design to load them explicitly.